Advanced Day 10 — Session 10 (Hours 37–40)

Python Programming (Advanced) • API Security, Python Clients, Integration, and Milestone Demo

Session 10 Overview

Topics Covered Today

- Hour 37 — API security basics: API key gate and safe storage
- Hour 38 — Consuming your own API with Python client functions
- Hour 39 — Integration choice: GUI talks to API or parallel UI/API
- Hour 40 — Checkpoint 5: API milestone demo

Source Alignment

- `Advanced/lessons/lecture/Day10_Hour1_Advanced.md` → `# Day 10, Hour 1: API security basics: API key gate + safe storage`
- `Advanced/lessons/lecture/Day10_Hour2_Advanced.md` → `# Day 10, Hour 2: Consuming your own API: Python client functions`

Session 10 Outcomes

By the end of today, learners will be able to:

- Gate write operations behind an environment-sourced API key using `X-API-Key`
- Read secrets from the environment rather than from source code
- Build a small `requests`-based Python client with timeouts and structured error handling
- Choose and defend one of two valid GUI/API integration architectures
- Deliver a rehearsed Checkpoint 5 demo covering CRUD, auth failure, and layer explanation

Scope Guardrails for Today

In Scope

- Simple API key gating for write operations (POST, PUT, DELETE)
- Reading secrets from the environment — not from source code
- Python client library wrapping `requests` with timeouts
- Two architectural integration patterns and the tradeoffs between them
- Checkpoint demo covering CRUD, consistent JSON errors, and API key protection

Not Yet

- Full OAuth, JWT, or identity systems
- Session management or cookie-based authentication
- Production-grade rate limiting or IP blocking

Hour 37: API Security Basics

Learning Outcomes

- Add a simple API-key gate for write operations
- Read secrets from the environment instead of source code
- Return 401 or 403 JSON errors consistently

Source Alignment

- `Advanced/lessons/lecture/Day10_Hour1_Advanced.md` → `## Learning Outcomes`

Vocabulary — Hour 37

- **API key gate** — a check at the HTTP boundary that blocks write operations when the correct secret is absent
- `X-API-Key` — the request header name conventionally used to carry the key
- `os.environ.get()` — reads from the process environment; the key lives outside of source code
- `hmac.compare_digest()` — timing-safe string comparison that prevents timing-oracle attacks
- **401 Unauthorized** — the key is missing or unacceptable
- **403 Forbidden** — the key is present but does not match (pick one convention and document it)

Source Alignment

Why the Key Must Not Live in Code

Secrets embedded in source code are:

- Visible in every Git clone, PR diff, and code review
- Hard to rotate without a code change
- Committed by accident more often than developers expect

Environment-based secrets are:

- Set per deployment without touching the codebase
- Rotatable without a commit
- Documented safely using placeholder examples only

> "Secrets do not belong in Git, screenshots, or hard-coded constants."
>
> — `Advanced/lessons/lecture/Day10_Hour1_Advanced.md` → `### Talk points`

Demo: `require_api_key` Decorator

```
import hmac
import os
from functools import wraps
from flask import request

def require_api_key(view_func):
    @wraps(view_func)
    def wrapper(*args, **kwargs):
        expected_key = os.environ.get("TRACKER_API_KEY")
        supplied_key = request.headers.get("X-API-Key")
        if not expected_key:
            return error_response("server_misconfigured",
                                  "API key is not configured", 500)
        if not supplied_key or not hmac.compare_digest(
                supplied_key, expected_key):
            return error_response("unauthorized",
                                  "Missing or invalid API key", 401)
        return view_func(*args, **kwargs)
    return wrapper
```

Demo: Applying the Decorator to a Write Route

```
@app.post("/records")
@require_api_key
def create_record():
    payload = request.get_json(silent=True)
    if payload is None:
        return error_response("invalid_json",
                              "Request body must be valid JSON.", 400)
    record = tracker_service.create_record(payload)
    return {"data": record.to_dict()}, 201
```

Three test cases to run during demo:

1. `POST /records` with **no** `X-API-Key` header — 401 JSON error
2. `POST /records` with **wrong** key — 401 JSON error
3. `POST /records` with **correct** key — 201 created record

Source Alignment

Hour 37 Lab — Protect the Write Surface

Time — 35–50 minutes

Tasks

1. Export `TRACKER_API_KEY` in the shell or run configuration
2. Add the `require_api_key` decorator (or equivalent helper)
3. Apply it to every `POST`, `PUT`, and `DELETE` route
4. Verify three cases manually: missing key, wrong key, correct key

Completion Criteria

- Write endpoints require the key; `GET` endpoints still work as intended
- Missing and wrong keys return JSON using the shared error contract
- No secret value is committed into source code

Common Pitfalls and Exit Ticket — Hour 37

Common pitfalls:

- Hard-coding the key value in `app.py` or any Python file
- Protecting `POST` but forgetting `PUT` or `DELETE`
- Returning Flask's default HTML error instead of JSON
- Printing or logging the actual secret value

Exit ticket — answer before moving on:

- What HTTP status code fits a missing API key in this design?
- Which file now owns the key-checking responsibility?
- How would a reviewer prove that the key check works without seeing the real key?

Source Alignment

Hour 38: Consuming Your Own API

Learning Outcomes

- Write a small Python client library using `requests`
- Set timeouts and convert HTTP failures into friendly messages
- Use the API key header for write operations without leaking the secret

Source Alignment

- `Advanced/lessons/lecture/Day10_Hour2_Advanced.md` → `## Learning Outcomes`

Vocabulary — Hour 38

- **API client** — a Python module that wraps raw HTTP calls in named functions
- **`timeout`** — maximum seconds `requests` will wait before raising `Timeout`
- **`raise_for_status()`** — converts any 4xx/5xx response into an `HTTPError`
- **`TrackerApiError`** — a custom exception carrying `status_code`, `code`, and `request_id`
- **Content-Type guard** — a check that confirms the response is JSON before calling `.json()`

Source Alignment

- `Advanced/lessons/lecture/Day10_Hour2_Advanced.md` → `### Talk points`

Why Build a Client Module?

A client module hides repetitive details so the rest of the app stays clean:

Without client module	With client module
`requests.get(...)` scattered everywhere	`client.list_records()`
Timeout missing on several calls	Timeout set once on the client object
Headers rebuilt per call site	`_headers()` method centralises them
`HTTPError` leaks into GUI/CLI code	`TrackerApiError` is the only exception

> "A timeout is not optional; without it the program can hang forever."
>
> — `Advanced/lessons/lecture/Day10_Hour2_Advanced.md` → `### Talk points`

Demo: `TrackerApiError` and `TrackerApiClient` Dataclass

```python
from dataclasses import dataclass
import requests

class TrackerApiError(RuntimeError):
    def __init__(self, message: str, *, status_code: int | None = None,
                 code: str | None = None,
                 request_id: str | None = None) -> None:
        super().__init__(message)
        self.status_code = status_code
        self.code = code
        self.request_id = request_id

@dataclass
class TrackerApiClient:
    base_url: str
    api_key: str | None = None
    timeout: float = 5.0

    def _headers(self) -> dict[str, str]:
        headers = {"Accept": "application/json"}
        if self.api_key:
            headers["X-API-Key"] = self.api_key
        return headers
```

Demo: `_request_json` — Central Error Translation

```python
def _request_json(self, method: str, path: str, **kwargs) -> dict | None:
    url = f"{self.base_url}{path}"
    try:
        response = requests.request(
            method, url, timeout=self.timeout,
            headers=self._headers(), **kwargs)
        response.raise_for_status()
        if response.status_code == 204 or not response.content:
            return None
        if "json" not in response.headers.get("Content-Type", "").lower():
            raise TrackerApiError(
                f"API returned non-JSON: {response.text[:200]}")
        return response.json()
    except requests.Timeout as exc:
        raise TrackerApiError(
            "API request timed out. Check that the server is running."
        ) from exc
    except requests.ConnectionError as exc:
        raise TrackerApiError(
            "Could not connect to the API. Check the base URL and port."
        ) from exc
```

Demo: `list_records` and `create_record`

```
    def list_records(self) -> list[dict]:
        payload = self._request_json("GET", "/records")
        if not payload:
            raise TrackerApiError("API returned no record data.")
        return payload["data"]

    def create_record(self, payload: dict) -> dict:
        response_payload = self._request_json(
            "POST", "/records", json=payload)
        if not response_payload:
            raise TrackerApiError("API returned no created record data.")
        return response_payload["data"]
```

Demo sequence:

1. Instantiate `TrackerApiClient` with `base_url` and `api_key`
2. Call `list_records()` — show the data list

Hour 38 Lab — Build the Client Module

Time — 35–50 minutes

Tasks

1. Create `client/api_client.py` with `TrackerApiClient`
2. Implement `list_records`, `create_record`, `update_record`, `delete_record`
3. Add `timeout` on every request path
4. Confirm write operations pass the `X-API-Key` header

Completion Criteria

- Client can list, create, update, and delete records end-to-end
- Timeouts are present on every request
- Write operations pass the API key header when configured

Common Pitfalls and Exit Ticket — Hour 38

Common pitfalls:

- No `timeout` argument on `requests` calls
- Forgetting headers on `PUT` / `DELETE` after adding them to `POST`
- Assuming every response body is JSON even for network failures
- Catching every exception and losing the useful status code

Exit ticket — answer before moving on:

- Why should a client set a timeout even during local development?
- What is the difference between `requests.Timeout` and `requests.HTTPError`?
- Which module is now the single place for all HTTP error translation?

Source Alignment

Hour 39: Integration Choice

Learning Outcomes

- Choose one capstone integration architecture and implement a cohesive flow
- Explain the tradeoff between GUI to API and parallel surfaces over one service layer
- Demonstrate both UI and API consistency over the same underlying data

Source Alignment

- `Advanced/lessons/lecture/Day10_Hour3_Advanced.md` → `## Learning Outcomes`

Vocabulary — Hour 39

- **Option A** — GUI calls `TrackerApiClient`; the API is the sole gateway to persistence
- **Option B** — GUI and API both call `TrackerService` and `TrackerRepository` directly
- **Schema drift** — the silent bug when GUI uses `name`, API uses `title`, database uses `item_name`
- **Single source of truth** — one layer (the service) owns the validation rule; every surface defers to it
- **Thin callback** — a GUI function that coordinates, but does not own, business rules

Source Alignment

- `Advanced/lessons/lecture/Day10_Hour3_Advanced.md` → `### Talk points`

Two Architecture Options

Option A — GUI calls API

```
Tkinter GUI
    ↓  TrackerApiClient  (HTTP + X-API-Key)
Flask API routes
    ↓  TrackerService
    ↓  TrackerRepository
    ↓  SQLite
```

Option B — Parallel surfaces, shared service

```
Tkinter GUI          Flask API routes
    ↓                     ↓
TrackerService  (shared)
    ↓
TrackerRepository → SQLite
```

Source Alignment

Option A Code Sketch — GUI Calls the API Client

```
from api_client import TrackerApiClient, TrackerApiError

def on_save_clicked() -> None:
    payload = {
        "title": title_var.get(),
        "category": category_var.get(),
        "status": status_var.get() or "open",
    }
    try:
        created = api_client.create_record(payload)
    except TrackerApiError as exc:
        messagebox.showerror("Save failed", str(exc))
        return
    refresh_table()
    save_status_var.set(f"Saved record {created['id']}")
```

The GUI never imports `TrackerService` or `TrackerRepository` directly.

Source Alignment

Option B Code Sketch — GUI Calls Service Directly

```python
from tracker.service import TrackerService, ValidationError

def save_directly_through_service() -> None:
    payload = {
        "title": title_var.get(),
        "category": category_var.get(),
        "status": status_var.get() or "open",
    }
    try:
        record = tracker_service.create_record(payload)
    except ValidationError as exc:
        messagebox.showerror("Save failed", str(exc))
        return
    save_status_var.set(f"Saved record {record.id}")
```

The API routes call the **same** `tracker_service` instance — no duplicate validation.

Source Alignment

Tradeoffs at a Glance

Consideration	Option A (GUI to API)	Option B (Parallel)
Complexity	More moving parts	Fewer layers
Failure points	Network errors to handle	Fewer error modes
Testability	GUI tests require running API	GUI tests use service directly
Schema alignment	Enforced by API contract	Requires field discipline
Demo risk	Higher if API is still unstable	Lower if API has rough edges

Rule of thumb: If your API is stable, choose Option A. If the API still has rough edges, Option B protects the milestone better.

Hour 39 Lab — Integration Sprint

Time — 35–50 minutes

Tasks

1. Choose Option A or B — state your choice in a code comment or README line
2. Implement one end-to-end action (create and list) through the chosen path
3. Prove both surfaces read the same database records
4. Confirm validation errors surface clearly to the user — no silent failures

Completion Criteria

- Chosen integration path works for at least create and list
- Learner can explain why they chose A or B
- Validation errors are shown consistently and do not crash the UI
- The same field-level state reaches GUI, API, and database

Common Pitfalls and Exit Ticket — Hour 39

Common pitfalls:

- Two schemas: `name` in GUI, `title` in API, `item_name` in the database
- GUI does not refresh the table after an API call completes
- Running the API and GUI against different SQLite database files
- Turning the GUI callback into a large block of networking code

Exit ticket — answer before moving on:

- What is the biggest tradeoff of your chosen architecture?
- Where is the single validation rule written that both surfaces must respect?
- What command proves both surfaces share the same underlying data?

Source Alignment

Hour 40: Checkpoint 5 — API Milestone Demo

Learning Outcomes

- Deliver a working REST API backed by SQLite
- Demonstrate CRUD, consistent JSON errors, and API-key protection
- Use a short rubric to identify final fixes before analytics and testing

Source Alignment

- `Advanced/lessons/lecture/Day10_Hour4_Advanced.md` → `## Learning Outcomes`

What the Checkpoint Must Prove

A milestone demo is **evidence**, not a sales pitch:

1. The API starts reliably from documented instructions
2. Health, list, create, update, and delete all work
3. Missing or wrong API keys produce a consistent JSON error
4. Persistence is real — data survives a server restart
5. Routes and service logic are visibly separated

> "Show the happy path first, then deliberately prove sad paths."
>
> — `Advanced/lessons/lecture/Day10_Hour4_Advanced.md` → `### Talk points`

Fast-Grade Demo Sequence

```
# Suggested demo order — run each step, read the result aloud

# 1. GET /health          -> 200 {"status": "ok"}
# 2. GET /records         -> 200 {"data": [...]}
# 3. POST /records        (no X-API-Key) -> 401 JSON error
# 4. POST /records        (with key) -> 201 created record
# 5. PUT  /records/<id>   (with key) -> 200 updated record
# 6. DELETE /records/<id> (with key) -> 204 or documented success
# 7. GET  /records/<id>   (after delete) -> 404 JSON error
```

Prove persistence: restart the server between steps 4 and 2, then re-run step 2 to show data survived.

Source Alignment

- `Advanced/lessons/lecture/Day10_Hour4_Advanced.md` → `### Demo code or command sketch`

Stabilization Checklist

Before queuing for a review, tick every item:

- [ ] API starts from a fresh terminal with `flask run` or `python -m api.app`
- [ ] `GET /health` returns JSON `{"status": "ok"}`
- [ ] `POST /records` with valid key returns `201` with the new record
- [ ] `PUT /records/<id>` with valid key returns `200` with updated data
- [ ] `DELETE /records/<id>` with valid key returns `204` or documented success
- [ ] Any invalid input returns `400` JSON using the shared error contract
- [ ] Missing or wrong key returns `401` or `403` JSON
- [ ] Data persists — a fresh `GET /records` after restart confirms it

Source Alignment

- `Advanced/lessons/lecture/Day10_Hour4_Advanced.md` → `### Completion criteria`

Capstone Quality Gate

Category	Ready evidence	Needs attention
Correctness	Feature works with a realistic happy path	Only works with hard-coded or instructor data
Error handling	Sad path produces clear JSON error or exception	App crashes or returns HTML traceback
Structure	Responsibility is in the correct layer	Route, service, and SQL logic tangled together
Repeatability	A command or documented step reproduces the result	Result depends on manual memory or IDE state
Maintainability	Names are clear; next feature has an obvious place	Code works once but will be hard to extend

Common Pitfalls and Exit Ticket — Hour 40

Common pitfalls:

- Returning an HTML error page during a sad-path demo
- Uncaught service exceptions causing `500` for predictable validation issues
- No visible separation between route logic and service logic
- Demo exceeds the time budget because commands were not rehearsed

Exit ticket — answer before moving on:

- What is one concrete design choice that kept your API maintainable?
- Which route would a reviewer probe first to test your security gate?
- Name the weakest part of your milestone honestly — and the single next fix

Source Alignment

Day 10 Wrap-Up

End-of-Day Evidence

By the end of this session, each capstone should have:

- Write routes protected by `X-API-Key` read from the environment
- A `TrackerApiClient` with timeout, `_headers`, and structured error translation
- A documented integration choice (Option A or Option B) with working create/list
- A rehearsed Checkpoint 5 demo covering the full fast-grade sequence

Session 10 Exit Ticket

Three questions to answer before leaving:

1. Which write route is most confidently protected — and which one still worries you?
2. What did writing the Python client reveal about your own API contract?
3. Which integration architecture did you choose, and what would make you reconsider it?

> "Your exit ticket is a sentence, not an essay: name what works, name what still needs attention, and name your next command."
>
> — `Advanced/lessons/lecture/Day10_Hour4_Advanced.md` → `## Reflection, Quick Checks, and Exit Ticket`

Looking Ahead — Day 11

Next Session

- Turn persisted tracker records into meaningful summaries with `pandas`
- Build report-ready charts using `matplotlib`
- Add an analytics layer as a real capstone deliverable
- Introduce a limited regression test workflow to protect growing complexity

> Today's API stability is tomorrow's analytics foundation. Checkpoint 5 is the gate.